

Distributed Systems - 10_11

What is an idea we can use to solve consensus?

Let's say we're in a synchronous system, and we want to decide a value. We could use a leader election, by having the leader decide the value.

So, for a system of 4 processes, we could have p_1 proposing 1, p_2 proposing 2, etc. Process p_1 decides his value immediately, and then sends a message to other people to let them know they have to decide that same value. The problem lies in the fact that p_1 could crash. However, as for the fact we are in a synchronous system, we could elect a new leader.

If, however, the message of deciding 1 reaches even one process, then that process will decide that value, and then other processes could decide other values (with a new leader), violating agreement.

How the algorithm we see solves this problem?

The idea is that we divide the algorithm in logical rounds. Remember that a round is a portion of time in the system we can map to an integer. A logical round is different from the concept of "round" seen before, however.

Let's have p_1 as the leader of round 1, deciding $v = 1$ and send that value to other people. When I receive the value sent by the leader I do not decide, but change opinion, so my proposal is changed with a new value. I decide when I'm the leader of the current round.

After some actions we'll reach round 2, and then p_2 will be the leader of that round.

The actions that bring us to a new round are given by receiving a message from the leader, or when I detect that the leader has crashed.

Process p_2 , even if he didn't receive the proposal from p_1 , will detect that p_1 has crashed, and be elected as the leader. Then he can propose a new value, and send it to others.

When we go to a new round, if the leader did not crash, then everyone will have the same value as proposal.

Let's take a look at the code:

Algorithm 5.2: Hierarchical Consensus	
Implements: Consensus, instance c .	
Uses: BestEffortBroadcast, instance beb ; PerfectFailureDetector, instance $\mathcal{P}$.	
upon event $\langle c, \text{Init} \rangle$ do $detectedranks := \emptyset$; $round := 1$; $proposal := \perp$; $proposer := 0$; $delivered := [\text{FALSE}]^N$; $broadcast := \text{FALSE}$;	upon $round = \text{rank}(self) \wedge proposal \neq \perp \wedge broadcast = \text{FALSE}$ do $broadcast := \text{TRUE}$; trigger $\langle beb, \text{Broadcast} \mid [\text{DECIDED}, proposal] \rangle$; trigger $\langle c, \text{Decide} \mid proposal \rangle$;
upon event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$ do $detectedranks := detectedranks \cup \{\text{rank}(p)\}$;	upon $round \in detectedranks \vee delivered[round] = \text{TRUE}$ do $round := round + 1$;
upon event $\langle c, \text{Propose} \mid v \rangle$ such that $proposal = \perp$ do $proposal := v$;	upon event $\langle beb, \text{Deliver} \mid p, [\text{DECIDED}, v] \rangle$ do $r := \text{rank}(p)$; if $r < \text{rank}(self) \wedge r > proposer$ then $proposal := v$; $proposer := r$; $delivered[r] := \text{TRUE}$;

In detectedranks we put ids of processes who died. In delivered we put the values sent by other processes to me (as boolean flag). proposal is my local value proposed, proposer is the id of the process who made me change idea.

The third handler is useful as in Consensus we can only propose one. If we want to start a new proposal, we have to make a new instantiation of the Consensus primitive.

The fourth handler checks if we are in the round in which we are the leader (the round number is equal to our id), our proposal shouldn't be $\perp$ and broadcast is False. We then set broadcast=True so that we can't broadcast again.

The last handler is executed when we receive a message. I take the id of the sender, and check that the id is smaller than ours (it's okay, as the order of leaders is the order of $\mathbb{N}$, also we discard messages coming from a subsequent round, as we already received the message). I also check that the received Id is greater than the previous proposer.

Even if we don't enter the if, we set $delivered[r] := \text{True}$. This is the action that makes me go to the new round. The last handler checks either if the process with corresponding round has crashed, or if we have decided the value coming from the leader, then we increase the round number.

This algorithm generates N^2 messages in the worst case. The message delay is N .

Let's take a look at the proofs:

- Integrity: How do we prove it formally? We decide at the fourth handler, which can't be executed more than once.

- Validity: We decide always in the fourth handler, and the value was either proposed from us (and then it's simple), or proposed by someone else. ...
- Termination: By induction on the id.
- Agreement. ...

- Integrity: There is only one line in which you decide. The line is protected by a boolean variable `Broadcast==False` that is set to `True` inside the handler. Therefore, it can be executed at most once.
- Validity: You can only decide the value of your proposal variable. Such a variable can be set on two lines:
 - In the propose handler -> this means that what proposed by yourself
 - In the `Broadcast` handler -> in this case I receive a proposal variable of someone else, so it was proposed by someone (see previous point).
- Termination: By induction on the process id.
 - Base case: `id=1` decides or crashes -> round 1 = `id=1` thus the handler used to decide is eventually executed or the process crashes.
 - Inductive hypothesis: process with `id=k-1` either crashes or terminate
 - Inductive case `id=k`. By inductive hypothesis all processes with ids in `1..k-1` either terminates or crash. This implies that the handler that increases the round will be true for all rounds in `1...k-1`. Therefore, eventually round will be `k`. At this point, as in the base case, either I decide or I crash.

- Agreement:
 - Let `p_i` be the correct process with minimum ID.
 - No process will go in round `i+1` without receiving the decided message by `p_i` (recall `p_i` is correct).
 - This means that for a process to update its round to `i+1` it has to put proposal to the same proposal of `p_i`.
 - Therefore each process with ID greater than `p_i` decides the same proposal of `p_i`. Recall, that you will not accept in round `i+1` messages from old leaders
 - The ID of `p_i` is the minimum so this set is the set contains all correct processes

We'll now take a look at uniform consensus. It has the same three properties as the regular one, plus another (agreement modified):

- (Uniform Agreement): No two processes decides differently

For the uniform consensus, no two processes decides differently, even if one is not correct. We've seen an execution when one process, like p_1 , decides 1 and then

crashes. Others could decide 2. We could modify this thing by introducing an ACK system, so that p_1 decides a value only when he has received an ACK.

When a correct process imposes a value and receives the ACKS, then everyone has set their value to the one proposed, and so the leader can decide and tell others to do, and it's safe as everyone has saved that value.

```

Uses:
  PerfectPointToPointLinks, instance pl;
  BestEffortBroadcast, instance beb;
  ReliableBroadcast, instance rb;
  PerfectFailureDetector, instance  $\mathcal{P}$ .

upon event  $\langle uc, Init \rangle$  do
  detectedranks :=  $\emptyset$ ;
  ackranks :=  $\emptyset$ ;
  round := 1;
  proposal :=  $\perp$ ; decision :=  $\perp$ ;
  proposed :=  $[\perp]^N$ ;

upon event  $\langle \mathcal{P}, Crash \mid p \rangle$  do
  detectedranks := detectedranks  $\cup$  {rank(p)};

upon event  $\langle uc, Propose \mid v \rangle$  such that proposal =  $\perp$  do
  proposal := v;

upon round = rank(self)  $\wedge$  proposal  $\neq \perp \wedge$  decision =  $\perp$  do
  trigger  $\langle beb, Broadcast \mid [PROPOSAL, proposal] \rangle$ ;

upon event  $\langle beb, Deliver \mid p, [PROPOSAL, v] \rangle$  do
  proposed[rank(p)] := v;
  if rank(p)  $\geq$  round then
    trigger  $\langle pl, Send \mid p, [ACK] \rangle$ ;

upon round  $\in detectedranks$  do
  if proposed[round]  $\neq \perp$  then
    proposal := proposed[round];
    round := round + 1;

upon event  $\langle pl, Deliver \mid q, [ACK] \rangle$  do
  ackranks := ackranks  $\cup$  {rank(q)};

upon detectedranks  $\cup$  ackranks = {1, ..., N} do
  trigger  $\langle rb, Broadcast \mid [DECIDED, proposal] \rangle$ ;

upon event  $\langle rb, Deliver \mid p, [DECIDED, v] \rangle$  such that decision =  $\perp$  do
  decision := v;
  trigger  $\langle uc, Decide \mid decision \rangle$ ;

```

In ackranks we put ACKs received by others.

We go to a new round only if the leader of the round crashes.

We use both beb and rb, depending on the case. The rb is used for the decision, ensuring every correct process receives the message of decision.

Let's see the proof:

- Integrity and Validity: The same proof of the non-uniform hierarchical
- Termination: By induction on the IDs, similar to the previous algorithm:
 - Base Case ID=1: , for the BebCast and the perfect failure detector if p_1 is correct it eventually executes the reliableBroadcast of Decided and delivers it terminating.
 - Inductive Hypothesis, ID=k-1: terminates or crashes
 - Inductive step ID=k: If a process with id less or equal to k-1 it is correct then it terminates by a ReliableDelivered of a Decided messages, for the property of the reliable broadcast if I am correct I will also receive a Decided message terminating. If all processes with ID less or equal to k-1 crash, then by the property of P I go to round $r=k$, since my ID is k I crash or terminate with the same prof of the base case.

- Uniform agreement: Let p_i be the first process that decides, then it received the acks from all correct process (each process has $\text{proposed}[i]=v$, where v is the value decided by p_i). We have two cases:
 - p_i is correct $\rightarrow$ the other processes receive the decided message from p_i , they have to decide the same value
 - p_i crashes $\rightarrow$ all the processes that go to round $i+1$ set $\text{proposal}=v$, the processes that decide in round i will decide by receiving the decided message of p_i that also contains v .

What is the message complexity of the algorithm?

I can express the complexity depending on the number of failures (f).

If we have zero failures, p_1 is correct, then we propose, receive the acks, and then we send with rb , which could take ? messages.

So, the message complexity is given by $(f + 1) \cdot (beb + rb)$.

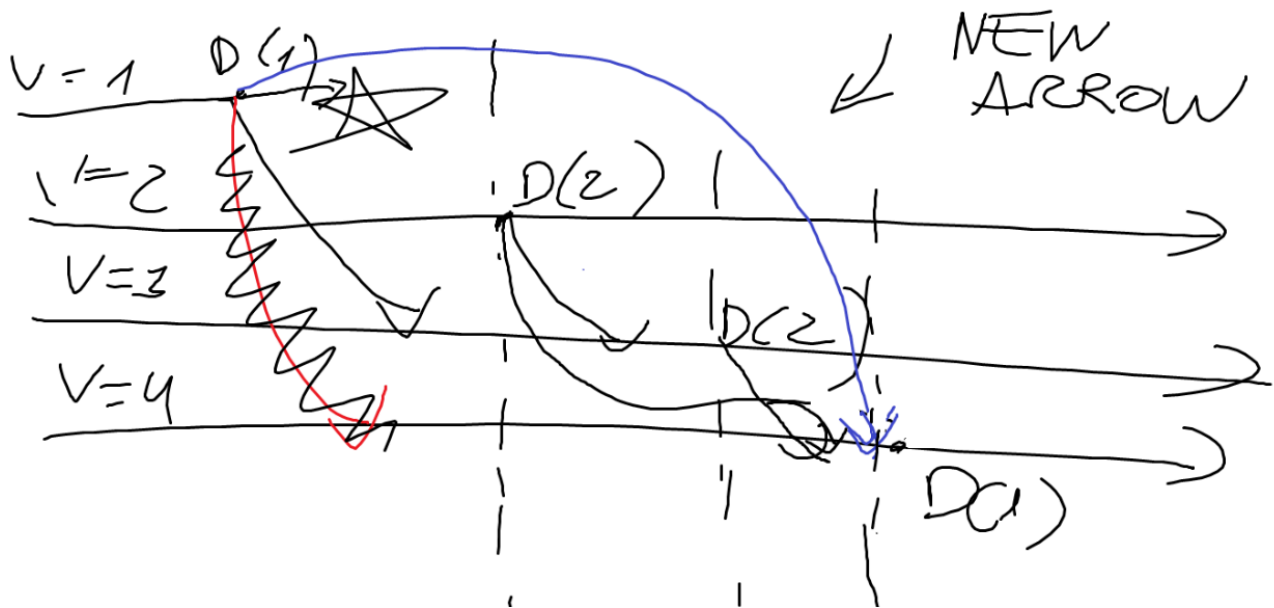
What is the message delay?

2 delays for each leader failing, 3 for each leader succeeds in sending and delivering, so it depends on f , and so $O(f)$. We will see that $O(f)$ is a lower bound. Every consensus algorithm has to have at least one execution where the message delay is $f + 1$.

Now, what happens if we remove the $r > \text{proposer}$ in the regular consensus? Is integrity violated? To violate it, a process has to decide twice. A process only decides in the fourth handler, so if I remove that line can I make someone Decide twice? No, as `broadcast` is set to `True` and so it's not a problem. Can we violate termination? No, if I modify the idea I change idea more times, am I changing the number of messages we are sending? No, and because we are not changing the two mechanism that lets us terminate, termination is not violated. Are we violating validity? No, we are not inventing messages by removing that line. So, we are probably violating agreement.

And to show this, we have to show an execution.

To do so, we need to receive a message from an old leader. Is it possible to create an execution without or with failures? I need a failure, and to do this we can try the easiest idea, having p_1 fail.



With these kinds of exercises, we check if the line we are modifying is touching some else breaking other properties, and if not we can check other properties if they are violated.

Let's try an exercise:

Uses:

PerfectPointToPointLinks, **instance** *pl*;
BestEffortBroadcast, **instance** *beb*;
ReliableBroadcast, **instance** *rb*;
PerfectFailureDetector, **instance** *P*.

upon event $\langle uc, Init \rangle$ **do**

detectedranks := $\emptyset$;
ackranks := $\emptyset$;
round := 1;
proposal := $\perp$; *decision* := $\perp$;
proposed := $[\perp]^N$;

upon event $\langle P, Crash \mid p \rangle$ **do**

detectedranks := *detectedranks* $\cup$ {rank(*p*)};

upon event $\langle uc, Propose \mid v \rangle$ **such that** *proposal* = $\perp$ **do**

proposal := *v*;

upon *round* = rank(*self*) $\wedge$ *proposal* $\neq \perp \wedge$ *decision* = $\perp$ **do**

trigger $\langle beb, Broadcast \mid [PROPOSAL, proposal] \rangle$;

upon event $\langle beb, Deliver \mid p, [PROPOSAL, v] \rangle$ **do**

proposed[rank(*p*)] := *v*;
if rank(*p*) $\geq$ *round* **then**
trigger $\langle pl, Send \mid p, [ACK] \rangle$;

upon *round* $\in$ *detectedranks* **do**

if *proposed*[*round*] $\neq \perp$ **then**
proposal := *proposed*[*round*];
round := *round* + 1;

upon event $\langle pl, Deliver \mid q, [ACK] \rangle$ **do**

ackranks := *ackranks* $\cup$ {rank(*q*)};

upon *detectedranks* $\cup$ *ackranks* = {1, ..., *N*} **do**

trigger $\langle rb, Broadcast \mid [DECIDED, proposal] \rangle$;

upon event $\langle rb, Deliver \mid p, [DECIDED, v] \rangle$ **such that** *decision* = $\perp$ **do**

decision := *v*;
trigger $\langle uc, Decide \mid decision \rangle$;

What happens if we use
Beb instead of Rb?

Integrity isn't touched, as well as validity, and agreement. Termination, however, is violated. The only way to create a failure of the termination, is having the process sending the message fails.

With two processes (*p*₁ failing), if we receive the message as process *p*₂ then there is no problem with termination. If I don't detect it, however, we'll detect the other process as failed and we'll terminate correctly.

Then we need at least three processes. We'll have two cases, one in which only *p*₂

receives the message, one where only p_3 receives it.

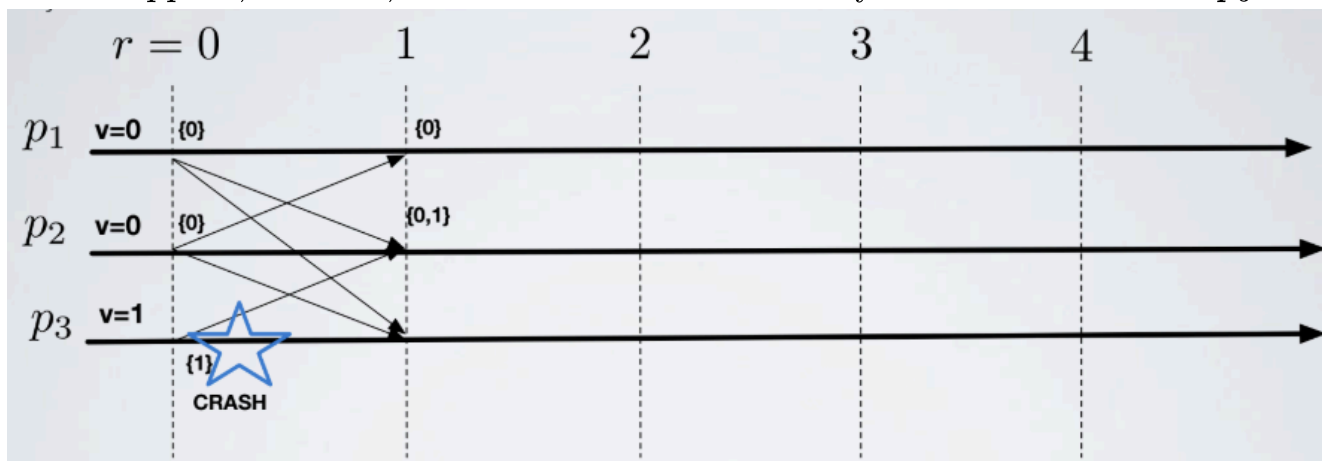
If p_2 receives it, then he decides the value. Then we detect the failure of p_1 , going from round 1 to 2. ...

Let's now take a look at the intuitive proof on the lower bound on time, so $O(f)$. Remember that there is an equivalence between synchronous setting and the concept of rounds.

Also, all the algorithms built in Distributed Computing can be built in synchronous setting using Full-information Protocol. We start at round 0, and send my memory to everyone. At round 1 I collect memory received from everyone, put it in mine, and send mine to everyone, and so on. With this pattern we can solve problems, so we send our memory to everyone. This pattern is obviously extremely expensive, having N^2 messages at each round.

Let's try to solve consensus with this pattern. At the beginning, the only value that matters we only have the propose. At round 0, everyone sends its propose to everyone. At the end, everyone will receive all the propose that are in the system, so in the example $\{0, 1\}$. It's impossible, if there are no failures, that sets are different. Now, how do I solve consensus? We could take the minimum or the maximum value in the set, and decide.

What happens, however, if there are failures? Let's say we have a failure for p_3 .



It's possible that only a subset of processes has received the value proposed from a crashed process, so sets will not align. In the example above we solved consensus anyways by taking the minimum. However, if p_1 and p_2 proposed 1, and p_3 0, then it would not work.

We could then broadcast again, and if there are no more crashes, then we'll have equal set. This costs, obviously, the addition of a new round.

So, for f rounds, we need $f + 1$ rounds to make this algorithm work.

If we do not use Ids in the decision, we can't do better than that.

So, let's introduce a Theorem: Given a synchronous system with f crash failures and n processes. There exists no algorithm that terminates in less than $f + 1$ rounds, and so there exists no algorithm that has a step complexity of f or less.